

PDD - PORTAL FAKE

Documento de Projeto de Desenvolvimento

Sistema: Portal Fake	Desafio: 05 - PDD com GitFlow
Versão: 1.0	Data: 06/07/2026

1. Descrição do Processo

Este documento descreve o processo de desenvolvimento do sistema **Portal Fake**, uma aplicação web fictícia para gerenciamento de usuários. O desenvolvimento será conduzido utilizando a metodologia **GitFlow** para controle de versão, simulando um ambiente de trabalho em equipe com múltiplos desenvolvedores.

O processo abrange desde a criação do repositório e definição da estrutura de branches até a implementação dos módulos, revisão de código via Pull Requests (PRs) e a geração de relatórios finais. O foco principal é garantir a qualidade do código, a organização do fluxo de trabalho e a rastreabilidade de cada alteração.

As branches principais são:

- main**: Contém o código em produção, estável e pronto para deploy.
- develop**: Branch de integração principal, onde as novas funcionalidades são mescladas antes de irem para a **main**.
- feature/***: Branches temporárias criadas a partir da **develop** para o desenvolvimento de novas funcionalidades ou correções específicas.

2. Objetivo do Processo

O objetivo principal deste processo é simular um ambiente de desenvolvimento de software real, utilizando boas práticas de engenharia de software e gestão de código.

1. **Praticar o Fluxo GitFlow:** Consolidar o conhecimento sobre a metodologia GitFlow.
2. **Simular Trabalho em Equipe:** Demonstrar a colaboração entre desenvolvedores através de Pull Requests.
3. **Garantir Qualidade:** Estabelecer um processo onde a qualidade do código é verificada antes de ser integrada.
4. **Documentar o Desenvolvimento:** Criar um registro completo para futuras manutenções e auditorias.
5. **Entregar um Sistema Funcional:** Desenvolver módulos com alta confiabilidade.

3. Fluxo de Desenvolvimento (Branches, Commits, PRs)

1. Criação da Branch `develop`

A partir da `main`, cria-se a branch `develop` para ser o ponto central de integração da equipe.

2. Criação da Branch de Funcionalidade (`feature/*`)

Cada nova tarefa gera uma branch nova. Comando: `git checkout -b feature/nome-da-funcionalidade develop`

3. Desenvolvimento e Commits

Os commits devem ser atômicos e seguir padrões descritivos, como:

```
[FEAT] Adiciona formulário de cadastro de usuários  
[FIX] Corrige validação de e-mail duplicado  
[DOC] Atualiza README com instruções de instalação
```

4. Publicação e Criação do Pull Request (PR)

Após testes locais, o código é enviado ao GitHub e um PR é aberto para a branch `develop`, contendo descrição detalhada das alterações.

5. Revisão e Aprovação

Um revisor analisa a qualidade, lógica e cobertura de testes antes de aprovar a mesclagem do código.

6. Merge na `develop`

Após aprovação, a feature é incorporada à `develop` e a branch temporária é removida.

7. Release e Merge na `main`

Periodicamente, cria-se uma branch `release/*` para ajustes finais e posterior merge na `main` com geração de tag de versão.

4. Regras de Negócio

- **Cadastro de Usuários:** Exige Nome, E-mail único, Senha forte (8+ chars, letra/número). ID gerado automaticamente.
- **Consulta de Usuários:** Listagem em tabela, filtros por nome/e-mail e paginação (10 por página).
- **Exportação de Dados:** Suporte a CSV e JSON com aplicação de filtros ativos.
- **Validação de Qualidade:** Testes automatizados obrigatórios e geração de relatório final detalhado.

5. Exceções e Falhas

Situação	Tratamento
E-mail já cadastrado	Mensagem: "Este e-mail já está em uso..."
Senha fraca	Listar requisitos não atendidos.
ID Inexistente	Erro 404: "Usuário não encontrado."
Conflito de Merge	Resolução manual local imediata.

6. Pré-condições

- **Ferramentas:** Git, VS Code (com extensões Python), Python 3.10+.
- **Repositório:** Criado no GitHub com acesso de colaborador à equipe.
- **Segurança:** Regras de proteção de branch ativas (Pull Requests obrigatórios).

7. Ambiente de Desenvolvimento

Componente	Especificação
Sistema Operacional	Windows 10/11, macOS, ou Linux
Controle de Versão	Git + GitHub
Linguagem	Python 3.10+
Ambiente Virtual	venv

8. Estrutura do Repositório

```
.  
├─ main (Tags: v1.0.0...)  
├─ develop  
├─ feature/ (e.g., feature/cadastro-usuarios)  
├─ release/ (e.g., release/v1.0.0)  
└─ hotfix/ (e.g., hotfix/corrige-bug-login)
```

Fim do Documento